

Programmazione

Stored procedure avanzate

Controllo di flusso, condizioni, cicli e variabili dentro le stored procedure.

Qui il database smette di essere solo una lista di query isolate

Dentro una stored procedure puoi trovare logica più ricca di una semplice sequenza lineare. Per questo si parla di versioni più avanzate.

Pensa a una ricetta con scelte e ripetizioni: se l'acqua bolle, aggiungi la pasta; se non bolle, aspetta. La procedura può fare qualcosa di simile con i dati.

Per esempio puoi incontrare:

- IF
- CASE
- cicli
- variabili locali

Cosa significa in pratica

Vuol dire che il database può prendere piccole decisioni da solo, ripetere un passaggio o scegliere un comportamento in base ai dati.

Questo può essere utile, ma va usato con equilibrio. **Troppa logica nascosta nel database può rendere il progetto più difficile da seguire.**

Un esempio mentale

Una procedura potrebbe controllare il totale di un ordine:

- se il totale supera una soglia, applica uno sconto
- altrimenti lascia il prezzo normale

Questa è logica, non solo lettura o scrittura di dati.

Il criterio più sano

Usa questa potenza quando porta chiarezza o affidabilità reali.

Se invece complica troppo la comprensione del sistema, probabilmente stai spostando troppa logica nel posto sbagliato.

:::tip Se una procedura avanzata diventa lunga da spiegare, forse merita di essere divisa o documentata meglio. :::

Cursori

Scorrere i risultati una riga alla volta quando un'elaborazione insieme non basta.

SQL ama lavorare per insiemi, ma non sempre basta

Un cursore permette di scorrere un risultato riga per riga. È come passare il dito lungo un elenco, fermandoti ogni volta su un elemento.

Di solito SQL preferisce lavorare su tante righe insieme. Il cursore è l'eccezione: prende una riga, la guarda, poi passa alla successiva.

Quando entra in scena

I cursori diventano utili in casi particolari, quando davvero hai bisogno di trattare i risultati uno alla volta.

Detto questo, SQL dà il meglio di sé quando ragiona su gruppi di righe insieme. **Per questo i cursori esistono, ma non sono lo strumento da preferire per tutto.**

Un esempio mentale

Se devi calcolare lo stesso aggiornamento per tutti i prodotti, una normale query `UPDATE` è spesso la scelta migliore.

Se invece devi chiamare una procedura diversa per ogni riga, con passaggi che dipendono dal risultato precedente, un cursore può avere senso.

La regola pratica

Se riesci a risolvere il problema con una query normale, in genere è meglio. Se il problema richiede una sequenza riga per riga, allora il cursore può avere senso.

Perché si consiglia prudenza

I cursori possono essere più lenti e più complessi da gestire rispetto alle operazioni insiemistiche tipiche di SQL.

Conoscerli è utile. Usarli come prima scelta quasi mai.

:::tip Prima di scegliere un cursore, prova a chiederti: "posso risolvere questo problema lavorando su tutte le righe insieme?" :::

Stored procedure

Blocchi di istruzioni salvati direttamente nel database per riutilizzare logica SQL.

Una stored procedure è una ricetta già pronta

Una stored procedure è un blocco di istruzioni salvato nel database. Invece di riscrivere sempre la stessa sequenza di comandi, puoi darle un nome e riutilizzarla.

È come salvare una ricetta con un titolo. Invece di riscrivere ogni volta tutti i passaggi, dici "prepara questa ricetta".

Un esempio piccolo

```
CREATE PROCEDURE aggiorna_prezzi ()
BEGIN
    UPDATE prodotti
    SET prezzo = prezzo * 1.10;
END;
```

Perché può essere utile

Una stored procedure è comoda quando:

- una sequenza di passaggi si ripete spesso
- vuoi centralizzare una logica vicino ai dati
- vuoi richiamare un'operazione complessa con un nome semplice

Un dettaglio importante

La sintassi cambia tra database diversi. **Il concetto però resta stabile:** salvi una procedura e poi la richiami quando ti serve.

:::caution Una procedura può modificare molti dati. Prima di usarla in un database reale, controlla bene cosa fa e su quali righe lavora. :::

Trigger

Azioni automatiche che scattano quando una tabella cambia.

Un trigger è una reazione automatica

Un trigger è una regola automatica. Succede un evento su una tabella, e il database reagisce.

Può scattare prima o dopo un `INSERT` , `UPDATE` o `DELETE` .

Un'immagine utile

Pensa a una porta con un sensore: appena qualcuno passa, succede qualcosa. Il trigger lavora in modo simile, ma sugli eventi del database.

Quando è utile

Può servire per:

- registrare modifiche in una tabella di log
- controllare certe regole automaticamente
- aggiornare dati collegati

Un esempio mentale

Immagina una tabella `prodotti` e una tabella `log_modifiche`. Ogni volta che cambia il prezzo di un prodotto, un trigger potrebbe registrare chi ha fatto la modifica e quando.

Così non devi ricordarti di scrivere la riga di log in ogni punto dell'applicazione.

Il lato da gestire con attenzione

I trigger possono essere potenti, ma anche un po' invisibili. Se un comportamento importante dipende da un trigger, chi legge il codice deve saperlo.

Per questo vanno usati con chiarezza e moderazione.

:::caution Se un trigger modifica altri dati senza che sia evidente, il sistema può diventare difficile da capire e da testare. :::

Funzioni definite dall'utente

Funzioni SQL create da te per restituire valori o risultati riutilizzabili.

Quando vuoi insegnare al database una piccola operazione nuova

Una funzione definita dall'utente permette di aggiungere una nuova funzione al tuo database. È simile a una stored procedure, ma in genere nasce per restituire un valore o un risultato.

Pensa a una calcolatrice con un tasto personalizzato. Premi quel tasto, gli dai un numero, e lei ti restituisce il risultato del calcolo che usi spesso.

Un esempio piccolo

```
CREATE FUNCTION prezzo_con_iva(prezzo DECIMAL(10,2))
RETURNS DECIMAL(10,2)
BEGIN
    RETURN prezzo * 1.22;
END;
```

Perché può tornare utile

Se hai un calcolo o una trasformazione che si ripete spesso, racchiuderla in una funzione può rendere il codice più ordinato.

Invece di riscrivere sempre la stessa formula, dai un nome a quell'idea.

Potresti poi usarla in una query:

```
SELECT nome, prezzo_con_iva(prezzo) AS prezzo_finale
FROM prodotti;
```

Qui `prezzo_con_iva` restituisce un valore calcolato per ogni prodotto.

Una distinzione semplice da ricordare

Non è una regola assoluta per tutti i database, ma puoi pensare così:

- la **procedura** esegue passi
- la **funzione** restituisce qualcosa

Questa scorciatoia mentale aiuta molto quando inizi.

:::note La sintassi delle funzioni cambia parecchio tra PostgreSQL, MySQL, SQL Server e altri database. L'idea di base, però, resta la stessa. :::